DOCUMENT 12 — CART & FAVORITES MANAGEMENT SPECIFICATION

1. Dokümanın Amacı

Bu doküman, Smart Chef & Pantry AI uygulamasında kullanıcıların:

- Tarifleri favorilerine eklemesini,
- Favorilerindeki tarifleri görüntülemesini,
- Favorilerden tarif kaldırmasını,
- Eksik malzemeleri alışveriş sepetine eklemesini,
- Sepetteki malzemeleri görüntülemesini,
- Malzeme miktarlarını artırıp azaltmasını,
- Sepetten malzeme silmesini,
- Sepeti tamamen temizlemesini

sağlayan **Favorites ve Cart modüllerinin teknik ve mimari gereksinimlerini** tanımlar.

Bu dokümanda özellikle veri sahipliği, yetkilendirme, miktar birleştirme, API davranışları, Flutter–Flask sorumlulukları, hata yönetimi, transaction kullanımı ve test senaryoları belirlenmektedir.

Bu modül, Document 01–11 içerisinde tanımlanan:

- PostgreSQL veritabanı,
- Flask backend,
- SQLAlchemy,
- Flutter Clean Architecture,
- Riverpod,
- REST API,
- JWT authentication,
- Recipe Matching & Recommendation Engine

yapılarıyla doğrudan uyumlu şekilde tasarlanmıştır.

---

2. Modül Kapsamı

Favorites ve Cart modülü iki ana parçadan oluşur:

2.1 Favorites

Kullanıcının beğendiği tarifleri kişisel olarak kaydetmesini sağlar.

Temel işlemler:

- Favoriye tarif ekleme
- Favoriden tarif çıkarma
- Favorileri listeleme
- Tarifin favori olup olmadığını öğrenme
- Aynı tarifin ikinci kez favoriye eklenmesini engelleme

2.2 Cart

Kullanıcının satın almak veya temin etmek istediği malzemeleri yönetmesini sağlar.

Temel işlemler:

- Malzeme ekleme
- Aynı malzemeyi birleştirme
- Miktar artırma
- Miktar azaltma
- Malzeme silme
- Sepeti listeleme
- Sepeti tamamen temizleme
- Tariften eksik malzemeleri sepete ekleme

---

3. Modülün MVP İçindeki Yeri

Favorites ve Cart modülleri MVP kapsamındadır.

MVP'de aşağıdaki özellikler bulunacaktır:

Özellik	MVP
Tarif favorileme	Evet
Favoriden çıkarma	Evet
Favorileri listeleme	Evet
Tarifin favori durumunu gösterme	Evet
Sepete malzeme ekleme	Evet
Aynı malzemeyi birleştirme	Evet
Miktar artırma/azaltma	Evet

Malzeme silme	Evet
Sepeti temizleme	Evet
Eksik malzemeleri sepete ekleme	Evet
Gerçek market siparişi	Hayır
Ödeme	Hayır
Gerçek zamanlı fiyat	Hayır
Market entegrasyonu	Hayır
Birden fazla alışveriş listesi	Hayır
Offline senkronizasyon	Hayır
Hatırlatıcı	Hayır

---

4. Aktörler

4.1 Guest

Giriş yapmamış kullanıcıdır.

Guest kullanıcı:

- Tarifleri görüntüleyebilir.
- Tarif önerilerini kullanabilir.
- Tarif detaylarını görebilir.
- Eksik malzemeleri görebilir.
- Ancak kişisel favori ve sepet verisini sunucuya kaydedemez.

Guest kullanıcı favori veya sepet işlemi yapmak istediğinde Login ekranına yönlendirilir.

4.2 Registered User

JWT ile authenticated olan kullanıcıdır.

Registered User:

- Favori ekleyebilir.
- Favori silebilir.

- Favorileri görüntüleyebilir.
- Sepete malzeme ekleyebilir.
- Sepetini görüntüleyebilir.
- Miktar değiştirebilir.
- Malzeme silebilir.
- Sepeti temizleyebilir.

4.3 Admin

Admin kullanıcılar MVP kapsamında normal kullanıcı özelliklerine ek olarak sistem yönetimi yetkilerine sahip olabilir.

Favorites ve Cart açısından admin:

- Kullanıcıların kişisel favorilerine doğrudan erişmemelidir.
- Kullanıcıların sepetlerini değiştirmemelidir.
- Normal kullanıcı gibi kendi favori ve sepetini yönetebilir.

Kullanıcı verilerine erişim yalnızca açıkça tanımlanmış yönetim gereksinimi olması halinde ayrıca tasarlanmalıdır.

---

5. Veri Modeli

Favorites ve Cart için Document 04'te tanımlanan iki tablo kullanılacaktır:

favorites
cart_items

---

6. Favorites Veri Modeli

6.1 favorites Tablosu

favorites
---
id UUID PRIMARY KEY
user_id UUID NOT NULL FK users.id
recipe_id UUID NOT NULL FK recipes.id
created_at TIMESTAMP NOT NULL

İlişkiler

User 1 ———————— N Favorite N ———————— 1 Recipe

Bir kullanıcı birçok tarifi favorileyebilir.

Bir tarif birçok kullanıcı tarafından favorilenebilir.

Ancak aynı kullanıcı aynı tarifi birden fazla kez favorileyemez.

---

7. Favorite Unique Constraint

Aynı kullanıcının aynı tarifi iki kez favorilemesini engellemek için:

UNIQUE(user_id, recipe_id)

kullanılmalıdır.

Örneğin:

user_id = A
recipe_id = R1

kaydı mevcutsa aynı kullanıcı için:

user_id = A
recipe_id = R1

ikinci kez oluşturulmamalıdır.

Bu kontrol yalnızca Flutter tarafında yapılmamalıdır.

Veritabanı seviyesinde de constraint bulunmalıdır.

Böylece aynı anda gelen iki istekte duplicate favorite oluşması engellenir.

---

8. Favorite İş Akışı

8.1 Tarif Favorileme

Kullanıcı Recipe Detail ekranında:

Favorite Button

butonuna basar.

Flutter:

POST /api/favorites

isteğini gönderir.

Backend:

1. JWT'yi doğrular.
2. Kullanıcı ID'sini JWT'den alır.
3. Recipe ID'nin geçerli olup olmadığını kontrol eder.
4. Tarifin mevcut olup olmadığını kontrol eder.
5. Tarif daha önce favorilenmiş mi kontrol eder.
6. Favorite kaydını oluşturur.
7. Başarılı response döner.

---

9. Favorite Silme

Kullanıcı favori butonuna tekrar bastığında:

DELETE /api/favorites/{recipe_id}

isteği gönderilir.

Backend:

1. JWT doğrulanır.
2. Current user belirlenir.
3. İlgili recipe ID ile favorite kaydı aranır.
4. Kayıt bulunursa silinir.
5. Başarılı response döndürülür.

Kullanıcının başka bir kullanıcının favorite kaydını silebilmesi mümkün olmamalıdır.

Silme sorgusu mantıksal olarak:

```
WHERE
   favorite.recipe_id = requested_recipe_id
```

AND favorite.user_id = current_user_id

şeklinde çalışmalıdır.

---

10. Favorite Listeleme

Endpoint:

GET /api/favorites

Authentication:

Required

Backend kullanıcıyı JWT'den belirler.

Request içerisinde:

user_id

gönderilmemelidir.

Örnek response:

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "favorite-uuid",
        "recipe": {
          "id": "recipe-uuid",
          "title": "Tavuklu Makarna",
          "image_url": "https://example.com/image.jpg",
          "category": "Ana Yemek"
        },
        "created_at": "2026-09-27T10:30:00Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
```

```
    "total": 1,
    "total_pages": 1
   }
  },
  "message": "Favorites retrieved successfully."
}
```


---

11. Favorite Durumu

Recipe Detail ekranında kullanıcının tarifi favorileyip favorilemediği gösterilmelidir.

Örneğin:

♡

→ Favoride değil.

♥

→ Favoride.

Bu bilgi backend'den alınmalıdır.

Frontend yalnızca görsel durumu yönetmemelidir.

---

12. Favorite Toggle

UI açısından favori işlemi:

Not Favorite
↓
Tap
↓
POST /favorites
↓
Favorite

ve:

Favorite
↓
Tap
↓
DELETE /favorites/{recipe_id}
↓
Not Favorite

şeklinde çalışmalıdır.

Backend gerçek veri kaynağıdır.

Flutter'daki optimistic UI yalnızca kullanıcı deneyimini iyileştirmek için kullanılabilir.

---

13. Cart Veri Modeli

Cart tablosu:

cart_items
---
id UUID PRIMARY KEY
user_id UUID NOT NULL FK users.id
ingredient_id UUID NOT NULL FK ingredients.id
quantity DECIMAL NOT NULL
unit VARCHAR NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL

İlişki:

User 1 ———— N CartItem N ———— 1 Ingredient

Bir kullanıcı birçok malzemeyi sepetinde tutabilir.

Bir malzeme birçok kullanıcının sepetinde bulunabilir.

---

14. Cart'ın Temel Kuralı

Cart içerisindeki aynı malzeme tekrar tekrar ayrı satır olarak oluşturulmamalıdır.

Örneğin kullanıcıda:

Domates x 2 adet

varsa ve tekrar:

Domates x 3 adet

eklenirse:

Domates x 5 adet

olmalıdır.

Yeni bir CartItem oluşturulmamalıdır.

---

15. Unit Kontrolü

Miktar birleştirme sırasında `unit` dikkate alınmalıdır.

Örneğin:

Domates - 2 adet
Domates - 3 adet

birleştirilebilir:

Domates - 5 adet

Ancak:

Domates - 2 kg
Domates - 3 adet

otomatik olarak:

5 ?

şeklinde birleştirilmemelidir.

Çünkü:

kg

ile:

adet

aynı fiziksel ölçüm değildir.

MVP'de farklı unit değerlerine sahip kayıtlar ayrı CartItem olarak tutulabilir.

Bu nedenle önerilen benzersizlik mantığı:

UNIQUE(user_id, ingredient_id, unit)

şeklindedir.

Unit `NULL` olduğunda backend'in tutarlı bir normalizasyon stratejisi uygulaması gerekir.

---

16. Cart'a Malzeme Ekleme

Endpoint:

POST /api/cart/items

Authentication:

Required

Request:

```
{
  "ingredient_id": "ingredient-uuid",
  "quantity": 2,
  "unit": "adet"
}
```

Backend:

1. JWT doğrulanır.

2. Current user alınır.
3. Ingredient ID kontrol edilir.
4. Quantity validation yapılır.
5. Unit normalize edilir.
6. Aynı kullanıcı + ingredient + unit kombinasyonu aranır.
7. Mevcut kayıt varsa quantity artırılır.
8. Kayıt yoksa yeni CartItem oluşturulur.
9. Response döndürülür.

---

17. Cart Merge Örneği

Mevcut sepet:

Domates 2 adet
Soğan 1 adet
Peynir 200 gram

Kullanıcı:

Domates 3 adet

ekler.

Sonuç:

Domates 5 adet
Soğan 1 adet
Peynir 200 gram

olmalıdır.

Yeni:

CartItem

oluşturulmaz.

---

18. Tariften Eksik Malzemeleri Sepete Ekleme

Bu özellik Recipe Detail ekranındaki:

Eksik Malzemeleri Sepete Ekle

aksiyonu ile çalışır.

Recipe Matching Engine kullanıcının eksik malzemelerini belirler.

Örneğin:

Tarif:
Tavuklu Makarna

Mevcut:
- Tavuk
- Makarna

Eksik:
- Krema
- Mantar
- Sarımsak

Kullanıcı:

Eksik Malzemeleri Sepete Ekle

butonuna basar.

Flutter backend'e gerekli malzemeleri gönderir veya API tasarımına göre her eksik malzeme için Cart API çağrısı gerçekleştirir.

Backend her malzeme için:

ingredient_id
quantity
unit

değerlerini işler.

Aynı malzeme kullanıcının sepetinde zaten varsa merge uygulanır.

---

19. Eksik Malzeme Miktarı

Recipe Matching Engine MVP'de quantity-aware matching yapmadığından, eksik malzemenin sepete eklenecek miktarı tarifte tanımlanan quantity üzerinden alınabilir.

Örneğin:

Krema — 200 ml

tarifte bulunuyorsa sepete:

Krema — 200 ml

olarak eklenebilir.

Ancak aynı ingredient zaten sepette varsa:

Krema — 100 ml

ile:

Krema — 200 ml

birleşerek:

Krema — 300 ml

olabilir.

MVP'de bu davranış quantity-aware recipe matching anlamına gelmez; yalnızca Cart tarafındaki miktar birleştirme işlemidir.

---

20. Cart Listeleme

Endpoint:

GET /api/cart

Authentication:

Required

Örnek response:


```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "cart-item-uuid",
        "ingredient": {
          "id": "ingredient-uuid",
          "name": "Domates",
          "category": "Sebze"
        },
        "quantity": 5,
        "unit": "adet",
        "created_at": "2026-09-27T10:00:00Z",
        "updated_at": "2026-09-27T10:30:00Z"
      }
    ],
    "total_items": 1
  },
  "message": "Cart retrieved successfully."
}
```


---

21. Cart Quantity Güncelleme

Endpoint:

PATCH /api/cart/items/{cart_item_id}

Request:


```
{
  "quantity": 5
}
```


Backend:

1. JWT doğrular.
2. CartItem'ı ID ile bulur.
3. CartItem'ın current user'a ait olduğunu kontrol eder.
4. Quantity validation yapar.
5. Quantity'yi günceller.
6. `updated_at` alanını günceller.
7. Güncel CartItem döndürür.

---

22. Quantity Kuralları

MVP'de quantity:

> 0

olmalıdır.

Örneğin:

0
-1
-5

geçersizdir.

Silme işlemi quantity'yi 0 yapmak yerine:

DELETE /api/cart/items/{cart_item_id}

ile gerçekleştirilmelidir.

Bu yaklaşım API davranışını daha net hale getirir.

---

23. Cart'tan Malzeme Silme

Endpoint:

DELETE /api/cart/items/{cart_item_id}

Backend yalnızca current user'a ait CartItem'ı silebilir.

Örnek:

DELETE /api/cart/items/abc-123

Başarılı işlem:

```
{
  "success": true,
  "data": null,
  "message": "Cart item removed successfully."
}
```

---

24. Sepeti Tamamen Temizleme

Endpoint:

DELETE /api/cart

Authentication:

Required

İşlem sonucunda yalnızca current user'a ait CartItem kayıtları silinir.

Başka kullanıcıların sepet verilerine dokunulmaz.

Örnek response:

```
{
  "success": true,
  "data": null,
  "message": "Cart cleared successfully."
}
```

Boş sepetin tekrar temizlenmesi hata olmak zorunda değildir.

İşlem idempotent davranacak şekilde tasarlanabilir.

---

25. Favorites API Specification

25.1 Listeleme

GET /api/favorites

Authentication:

Bearer JWT

25.2 Favori Ekleme

POST /api/favorites

Request:

```
{
  "recipe_id": "recipe-uuid"
}
```

Response:

```
{
  "success": true,
  "data": {
    "id": "favorite-uuid",
    "recipe_id": "recipe-uuid",
    "created_at": "2026-09-27T10:30:00Z"
  },
  "message": "Recipe added to favorites."
}
```

25.3 Favoriden Çıkarma

DELETE /api/favorites/{recipe_id}

Response:

```
{
  "success": true,
  "data": null,
  "message": "Recipe removed from favorites."
}
```

---

26. Cart API Specification

Listeleme

GET /api/cart

Malzeme Ekleme

POST /api/cart/items

Request:

```
{
  "ingredient_id": "ingredient-uuid",
  "quantity": 2,
  "unit": "adet"
}
```

Miktar Güncelleme

PATCH /api/cart/items/{cart_item_id}

Request:

```
{
  "quantity": 4
}
```

Malzeme Silme

DELETE /api/cart/items/{cart_item_id}

Sepeti Temizleme

DELETE /api/cart

---

27. Authentication ve Ownership

Favorites ve Cart tamamen kullanıcıya özel verilerdir.

Bu nedenle:

JWT → current_user_id

ilişkisi zorunludur.

Client tarafından:

```
{
  "user_id": "..."
}
```

gönderilmemelidir.

Örneğin kötü yaklaşım:

GET /api/cart?user_id=123

yerine:

```
GET /api/cart
Authorization: Bearer <JWT>
```

kullanılmalıdır.

Backend:

```
JWT
↓
current_user_id
↓
CartRepository
↓
WHERE user_id = current_user_id
```

mantığıyla çalışmalıdır.

---

28. Ownership Kontrolü

Bir CartItem'ın ID'sini bilen başka bir kullanıcı:

PATCH /api/cart/items/{id}

göndererek o kaydı değiştirememelidir.

Aynı şekilde:

DELETE /api/cart/items/{id}

ile başka kullanıcının kaydı silinememelidir.

Repository sorguları current user kontrolünü içermelidir.

Örnek mantık:

```
get_cart_item(
    cart_item_id,
    user_id
)
```

şeklinde tasarlanabilir.

Bu güvenlik kontrolü yalnızca Flutter tarafına bırakılmamalıdır.

---

29. Guest Kullanıcı Davranışı

Guest kullanıcı:

Recipe Detail
↓
Favorite
↓
Login Required
↓
Google Login
↓
Backend JWT
↓
Recipe Detail
↓
Favorite

akışını yaşamalıdır.

Aynı yapı Cart için de uygulanır:

Recipe Detail
↓
Add to Cart
↓
Login Required
↓
Google Login
↓
Cart işlemi

Login sonrasında kullanıcının daha önce bulunduğu ekran ve yapmak istediği işlem mümkün olduğunca korunmalıdır.

---

30. Flutter Architecture

Favorites feature:

```
features/
└── favorites/
    ├── data/
    │   ├── datasources/
    │   │   └── favorites_remote_data_source.dart
    │   ├── models/
    │   │   └── favorite_model.dart
    │   └── repositories/
    │       └── favorites_repository_impl.dart
    │
    ├── domain/
    │   ├── entities/
    │   │   └── favorite.dart
    │   ├── repositories/
    │   │   └── favorites_repository.dart
    │   └── usecases/
    │       ├── get_favorites.dart
    │       ├── add_favorite.dart
    │       └── remove_favorite.dart
    │
    └── presentation/
        ├── pages/
        │   └── favorites_page.dart
        ├── widgets/
        │   ├── favorite_recipe_card.dart
```

```
│   └── favorite_button.dart
└── providers/
    └── favorites_provider.dart
```

---

31. Cart Flutter Architecture

```
features/
└── cart/
    ├── data/
    │   ├── datasources/
    │   │   └── cart_remote_data_source.dart
    │   ├── models/
    │   │   └── cart_item_model.dart
    │   └── repositories/
    │       └── cart_repository_impl.dart
    │
    ├── domain/
    │   ├── entities/
    │   │   └── cart_item.dart
    │   ├── repositories/
    │   │   └── cart_repository.dart
    │   └── usecases/
    │       ├── get_cart.dart
    │       ├── add_cart_item.dart
    │       ├── update_cart_item.dart
    │       ├── remove_cart_item.dart
    │       └── clear_cart.dart
    │
    └── presentation/
        ├── pages/
        │   └── cart_page.dart
        ├── widgets/
        │   ├── cart_item_card.dart
        │   ├── quantity_selector.dart
        │   └── empty_cart.dart
        └── providers/
            └── cart_provider.dart
```

---

32. Flutter Data Flow

Favorites:

FavoritesPage
↓
FavoritesProvider
↓
GetFavorites
↓
FavoritesRepository
↓
FavoritesRemoteDataSource
↓
Dio
↓
Flask API

Cart:

CartPage
↓
CartProvider
↓
UseCase
↓
CartRepository
↓
CartRemoteDataSource
↓
Dio
↓
Flask API

UI doğrudan Dio kullanmamalıdır.

---

33. Favorite Button Architecture

Recipe Detail ekranındaki favorite butonu:

RecipeDetailPage
↓
FavoriteButton
↓
FavoritesProvider
↓
AddFavorite / RemoveFavorite

↓
Repository
↓
API

şeklinde çalışmalıdır.

FavoriteButton'ın backend ile doğrudan konuşması doğru değildir.

---

34. Cart Quantity Selector

Cart Item içerisinde:

[-] 2 [+]

kontrolü bulunabilir.

+:

quantity + 1

-:

quantity - 1

işlemi yapar.

Quantity:

1

iken - işlemi sonucunda:

0

yapmak yerine UI:

Remove

işlemini tetikleyebilir.

Alternatif olarak kullanıcıya silme onayı gösterilebilir.

Ancak backend'de:

quantity = 0

geçerli bir CartItem olarak tutulmamalıdır.

---

35. Optimistic UI

Favorites ve Cart işlemlerinde optimistic UI kullanılabilir.

Örneğin kullanıcı favorite butonuna bastığında ikon anında değişebilir:

♡ → ♥

Arka planda:

POST /api/favorites

gönderilir.

API başarısız olursa UI eski durumuna döndürülmelidir.

Örneğin:

♡
↓
♥
↓
API Error
↓
♡

Ancak optimistic UI hiçbir zaman backend doğrulamasının yerine geçmemelidir.

Backend gerçek veri kaynağıdır.

---

36. Server Truth

API başarılı olduğunda frontend state güncellenir.

API başarısız olduğunda:

- UI rollback yapılır.
- Kullanıcıya hata mesajı gösterilir.
- Gerekirse veri yeniden çekilir.

Özellikle:

favorite
cart quantity
cart merge

gibi işlemlerde server state ile Flutter state'in senkron kalması önemlidir.

---

37. Loading State

Favorites:

FavoritesLoading
FavoritesLoaded
FavoritesEmpty
FavoritesError

Cart:

CartLoading
CartLoaded
CartEmpty
CartError

gibi durumlarla yönetilebilir.

Örneğin Favorites ekranında:

Loading
↓
API Success
↓
Favorites

veya:

Loading
↓
API Success
↓
No Favorites

akışları bulunmalıdır.

---

38. Empty State — Favorites

Kullanıcının favorisi yoksa boş ekran bırakılmamalıdır.

Örnek:

Henüz favori tarifiniz yok.

Beğendiğiniz tarifleri favorilerinize
ekleyerek burada kolayca bulabilirsiniz.

Altında:

Tarifleri Keşfet

butonu bulunabilir.

---

39. Empty State — Cart

Sepet boşsa:

Sepetiniz boş.

Tariflerden eksik malzemeleri
buraya ekleyebilirsiniz.

ve:

Tarifleri Keşfet

butonu gösterilebilir.

---

40. Error State

Örneğin favorite ekleme başarısız olduğunda:

Tarif favorilere eklenemedi.
Lütfen tekrar deneyin.

Cart işlemi başarısız olduğunda:

Malzeme sepete eklenemedi.
Lütfen tekrar deneyin.

Backend'in teknik hata mesajları doğrudan kullanıcıya gösterilmemelidir.

Örneğin:

IntegrityError at PostgreSQL...

gibi mesajlar UI'a gönderilmemelidir.

---

41. Backend Architecture

Favorites:

routes/favorite_routes.py
↓
schemas/favorite.py
↓
services/favorite_service.py
↓
repositories/favorite_repository.py
↓
models/favorite.py
↓
PostgreSQL

Cart:

routes/cart_routes.py
↓
schemas/cart.py
↓
services/cart_service.py
↓
repositories/cart_repository.py
↓
models/cart_item.py
↓
PostgreSQL

---

42. Favorite Service Responsibilities

`FavoriteService`:

- Favorite oluşturma
- Duplicate kontrolü
- Favorite silme
- Favorite listeleme
- Recipe existence kontrolü
- Kullanıcı ownership kontrolü
- Business rule uygulama

Repository:

- SQLAlchemy sorguları
- Insert
- Delete
- Select
- Exists

işlemlerinden sorumludur.

Repository business logic taşımamalıdır.

---

43. Cart Service Responsibilities

`CartService`:

- Cart item ekleme
- Duplicate/merge kontrolü
- Quantity validation
- Unit kontrolü
- Cart item güncelleme
- Cart item silme
- Cart temizleme
- Kullanıcı ownership kontrolü
- Transaction yönetimi

işlemlerinden sorumludur.

---

44. Cart Merge Business Logic

Temel algoritma:

Input:
user_id
ingredient_id
quantity
unit

↓

Validate ingredient

↓

Validate quantity

↓

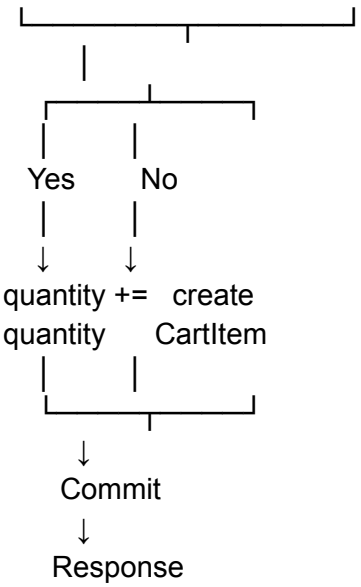
Normalize unit

↓

Find existing item
(user_id + ingredient_id + unit)

↓

┌──────────────┐
│ Existing? │



---

45. Transaction Yönetimi

Cart'a malzeme ekleme işleminde:

find existing
+
update/create
+
commit

aynı transaction kapsamında gerçekleştirilmelidir.

Favorite oluşturma da:

validate
+
create
+
commit

mantığında transaction içerisinde çalışmalıdır.

Transaction başarısız olursa ilgili işlem rollback edilmelidir.

---

46. Concurrency

Aynı anda iki request gelmesi durumunda:

POST /api/cart/items

işlemlerinin duplicate CartItem oluşturması engellenmelidir.

Örneğin iki request aynı anda:

Domates + 2

gönderirse sonuç:

Domates + 4

olmalıdır.

Bunun için yalnızca:

SELECT → INSERT

mantığına güvenilmemelidir.

Database unique constraint ve uygun transaction/locking/upsert yaklaşımı kullanılmalıdır.

Aynı durum Favorites için de geçerlidir.

---

47. Idempotency

Favorite delete:

DELETE /api/favorites/{recipe_id}

işlemi tekrar gönderildiğinde sistemin tutarlı kalması gerekir.

Cart clear:

DELETE /api/cart

boş sepet üzerinde tekrar çalıştırıldığında sistem hata vermek zorunda değildir.

Ancak:

POST /api/favorites

aynı kayıt için tekrar gönderildiğinde duplicate oluşturulmamalıdır.

Bu nedenle database constraint kritik öneme sahiptir.

---

48. API Error Codes

Favorites için:

AUTH_REQUIRED
INVALID_TOKEN
TOKEN_EXPIRED
RECIPE_NOT_FOUND
FAVORITE_NOT_FOUND
FAVORITE_ALREADY_EXISTS
FORBIDDEN
VALIDATION_ERROR
DATABASE_ERROR
INTERNAL_SERVER_ERROR

Cart için:

AUTH_REQUIRED
INVALID_TOKEN
TOKEN_EXPIRED
INGREDIENT_NOT_FOUND
CART_ITEM_NOT_FOUND
INVALID_CART_QUANTITY
VALIDATION_ERROR
FORBIDDEN
DATABASE_ERROR
INTERNAL_SERVER_ERROR

---

49. Favorite Duplicate Response

Kullanıcı zaten favoride olan bir tarifi tekrar eklerse:

HTTP:

409 Conflict

Response:

```
{
  "success": false,
  "error": {
    "code": "FAVORITE_ALREADY_EXISTS",
    "message": "Recipe is already in favorites.",
    "details": {}
  }
}
```

---

50. Favorite Not Found

Silinmek istenen favorite bulunamazsa:

HTTP:

404 Not Found

Response:

```
{
  "success": false,
  "error": {
    "code": "FAVORITE_NOT_FOUND",
    "message": "Favorite recipe was not found.",
    "details": {}
  }
}
```

---

51. Cart Item Not Found

```
{
  "success": false,
```

```
  "error": {
    "code": "CART_ITEM_NOT_FOUND",
    "message": "Cart item was not found.",
    "details": {}
  }
}
```

HTTP:

404

---

52. Invalid Cart Quantity

Örneğin:

```
{
  "quantity": -2
}
```

gönderilirse:

400 Bad Request

veya validation tasarımına göre:

422 Unprocessable Entity

döndürülebilir.

Response:

```
{
  "success": false,
  "error": {
    "code": "INVALID_CART_QUANTITY",
    "message": "Cart quantity must be greater than zero.",
    "details": {}
  }
}
```

---

53. Recipe Matching ile Cart İlişkisi

Cart modülü Recipe Matching Engine'in kendisini yönetmez.

Recipe Matching Engine:

User Ingredients
↓
Recipe Matching
↓
Matched Ingredients
↓
Missing Ingredients

oluşturur.

Cart ise:

Missing Ingredients
↓
Cart Items

dönüşümünü gerçekleştirir.

Dolayısıyla sorumluluklar ayrıdır.

RecommendationEngine
↓
MissingIngredient
↓
CartService
↓
CartItem

---

54. Recipe Detail → Cart Akışı

Tam akış:

Recipe Detail
↓
Missing Ingredients
↓
User selects:

"Add Missing Ingredients"
↓
Cart Use Case
↓
Cart Repository
↓
POST /api/cart/items
↓
CartService
↓
Merge / Create
↓
PostgreSQL
↓
Updated Cart
↓
Flutter State

---

55. Favorites → Recipe Detail

Favorites ekranında kullanıcı bir tarife tıklarsa:

Favorites
↓
Favorite Recipe Card
↓
Recipe Detail

yönlendirmesi yapılmalıdır.

Recipe Detail tekrar açıldığında:

- Tarif bilgisi
- Match percentage gerekiyorsa ilgili bağlam
- Favorite state
- Missing ingredients
- Cart actions

görüntülenebilir.

---

56. Favorites Sayfası

Önerilen ekran:

AppBar
"Favorilerim"

↓

Favorite Recipe Card



↓

Favorite Recipe Card
...

Her kart:

- Image
- Title
- Category
- Favorite button

içerebilir.

---

57. Cart Sayfası

Önerilen ekran:

AppBar
"Alışveriş Listem"

[Sepeti Temizle]

Domates
[-] 5 adet [+]

Soğan
[-] 2 adet [+]

Peynir
[-] 200 gram [+]

Her CartItem:

- Ingredient name
- Quantity
- Unit
- Increment
- Decrement
- Remove

işlemlerini desteklemelidir.

---

58. Provider State Yönetimi

Riverpod tarafında Cart için:

cartProvider

ve Favorites için:

favoritesProvider

kullanılabilir.

Provider:

- Loading
- Data
- Error

durumlarını yönetmelidir.

Business logic provider içine gereğinden fazla doldurulmamalıdır.

Örneğin:

cartProvider

doğrudan SQL veya Dio ayrıntılarını bilmemelidir.

---

59. Use Case Örnekleri

Favorites:

GetFavorites
AddFavorite
RemoveFavorite

Cart:

GetCart
AddCartItem
UpdateCartItem
RemoveCartItem
ClearCart

Recipe Detail'den:

AddMissingIngredientsToCart

gibi bir üst seviye use case de kullanılabilir.

Bu use case birden fazla Cart işlemine ihtiyaç duyuyorsa bunu kontrollü şekilde orkestre etmelidir.

---

60. Data Model — Flutter

Favorite entity:

Favorite
- id

- recipe
- createdAt

CartItem entity:

CartItem
- id
- ingredient
- quantity
- unit
- createdAt
- updatedAt

API model ile domain entity birbirinin aynısı olmak zorunda değildir.

Örneğin:

FavoriteModel

API JSON dönüşümünden sorumludur.

Favorite

ise domain tarafındaki iş nesnesidir.

---

61. Cache Yaklaşımı

MVP'de Favorites ve Cart için temel server-driven yaklaşım kullanılmalıdır.

Uygulama açıldığında:

Authenticated
↓
Fetch Favorites
Fetch Cart

yapılabilir.

Local cache gelecekte eklenebilir.

MVP'de local cache nedeniyle server state ile client state arasında karmaşıklık oluşturulmaması tercih edilir.

---

62. Offline Kullanım

Offline cart/favorites MVP kapsamında değildir.

İleride:

Local Cart
↕
Server Cart

senkronizasyonu yapılabilir.

Ancak offline sistem eklenecekse:

- conflict resolution
- last updated time
- deleted records
- merge strategy
- authentication state

gibi konular ayrıca tasarlanmalıdır.

---

63. Test Strategy

Favorites ve Cart modülleri için:

Unit Tests
Integration Tests
Widget Tests
API Tests

uygulanmalıdır.

---

64. Favorite Unit Tests

Test edilmesi gerekenler:

Test 1 — Favorite Add

Valid user
+
Valid recipe
=
Favorite created

Test 2 — Duplicate Favorite

Existing favorite
+
Same recipe
=
FAVORITE_ALREADY_EXISTS

Test 3 — Favorite Remove

Existing favorite
=
Deleted

Test 4 — Favorite Not Found

Non-existing favorite
=
FAVORITE_NOT_FOUND

Test 5 — Ownership

User A
+
User B's favorite ID
=
Access denied / not found

Test 6 — Authentication

No JWT
=
AUTH_REQUIRED

---

65. Cart Unit Tests

Test 1 — Add New Ingredient

Empty cart
+
Tomato 2 adet
=
Tomato 2 adet

Test 2 — Merge Same Ingredient

Tomato 2 adet
+
Tomato 3 adet
=
Tomato 5 adet

Test 3 — Different Unit

Tomato 2 adet
+
Tomato 500 gram
=
Two separate compatible records

Test 4 — Quantity Update

Tomato 2
→
Tomato 5

Test 5 — Quantity Validation

quantity = 0
=
INVALID_CART_QUANTITY

Test 6 — Negative Quantity

quantity = -1
=
INVALID_CART_QUANTITY

Test 7 — Remove

Existing CartItem
=
Deleted

Test 8 — Clear

Cart with N items
→
Empty cart

Test 9 — Ownership

User A
+
User B's cart item
=
Access denied / not found

Test 10 — Invalid Ingredient

Invalid ingredient_id
=
INGREDIENT_NOT_FOUND

---

66. Integration Test — Recipe to Cart

Aşağıdaki senaryo uçtan uca test edilmelidir:

User Login
↓
Recipe Recommendation
↓
Recipe Detail
↓
Missing Ingredients
↓
Add Missing Ingredients
↓
Cart
↓
Verify Items

Örneğin:

Missing:
- Domates 2 adet
- Soğan 1 adet

işlem sonrası:

Cart:
- Domates 2 adet
- Soğan 1 adet

olmalıdır.

---

67. Integration Test — Existing Cart Merge

Başlangıç:

Cart:
Domates 2 adet

Tarif eksik malzemesi:

Domates 3 adet

Eklendiğinde sonuç:

Cart:
Domates 5 adet

olmalıdır.

---

68. Widget Tests

Favorites:

- Empty state gösteriliyor mu?
- Favorite card gösteriliyor mu?
- Favorite button doğru çalışıyor mu?
- Loading state gösteriliyor mu?
- Error state gösteriliyor mu?

Cart:

- Empty cart gösteriliyor mu?
- Cart item gösteriliyor mu?
- Plus çalışıyor mu?
- Minus çalışıyor mu?
- Remove çalışıyor mu?
- Clear cart çalışıyor mu?
- Loading/error state doğru gösteriliyor mu?

test edilmelidir.

---

69. API Contract Testleri

Aşağıdaki endpointlerin contract testleri bulunmalıdır:

GET /api/favorites
POST /api/favorites
DELETE /api/favorites/{recipe_id}

GET /api/cart
POST /api/cart/items
PATCH /api/cart/items/{cart_item_id}
DELETE /api/cart/items/{cart_item_id}
DELETE /api/cart

Her endpoint için:

- status code
- response structure
- error structure
- authentication
- validation

kontrol edilmelidir.

---

70. Security Requirements

Favorites ve Cart için:

Zorunlu

- JWT doğrulaması
- Current user ID'nin JWT'den alınması
- User ownership kontrolü
- SQLAlchemy parameterized queries
- Input validation
- Quantity validation
- Database constraints
- Error response sanitization
- Secret bilgilerin loglanmaması

Kesinlikle yapılmamalı

Client tarafından user_id'ye güvenmek

veya:

Frontend'de ownership kontrolü yapıp backend'de kontrol etmemek

---

71. Database Constraints

Favorites:

PRIMARY KEY(id)

FOREIGN KEY(user_id)

FOREIGN KEY(recipe_id)

UNIQUE(user_id, recipe_id)

Cart:

PRIMARY KEY(id)

FOREIGN KEY(user_id)

FOREIGN KEY(ingredient_id)

CHECK(quantity > 0)

UNIQUE(user_id, ingredient_id, unit)

Unit nullable ise PostgreSQL'deki NULL unique davranışı ayrıca dikkate alınmalı ve uygulama seviyesinde normalize edilmelidir.

---

72. Indexler

Favorites:

INDEX favorites_user_id_idx
INDEX favorites_recipe_id_idx

ve unique constraint:

(user_id, recipe_id)

Cart:

INDEX cart_items_user_id_idx
INDEX cart_items_ingredient_id_idx

ve:

(user_id, ingredient_id, unit)

üzerinde uygun unique yapı bulunmalıdır.

Bu indexler özellikle kullanıcı bazlı listeleme ve ownership sorgularını hızlandırır.

---

73. Repository API Tasarımı

FavoriteRepository:

create(user_id, recipe_id)

exists(user_id, recipe_id)

get_by_recipe(user_id, recipe_id)

get_all(user_id, page, limit)

delete(user_id, recipe_id)

CartRepository:

get_all(user_id)

get_by_id(user_id, cart_item_id)

```
get_by_ingredient_and_unit(
    user_id,
    ingredient_id,
    unit
)
```

create(...)

update(...)

delete(...)

clear(user_id)

Repository metodlarında user ownership parametresinin bulunması güvenlik açısından faydalıdır.

---

74. Service API Tasarımı

FavoriteService:

add_favorite(user_id, recipe_id)

remove_favorite(user_id, recipe_id)

get_favorites(user_id, page, limit)

CartService:

```
get_cart(user_id)

add_item(
    user_id,
    ingredient_id,
    quantity,
    unit
)

update_item(
    user_id,
    cart_item_id,
    quantity
)

remove_item(
    user_id,
    cart_item_id
)

clear_cart(user_id)
```

RecipeService veya ayrı bir orchestration service:

```
add_missing_recipe_ingredients_to_cart(
    user_id,
    recipe_id
)
```

gibi bir işlem sağlayabilir.

---

75. API Layer Responsibilities

`favorite_routes.py`:

```
Request
↓
JWT
↓
Schema validation
↓
FavoriteService
↓
```

Response

`cart_routes.py`:

Request
↓
JWT
↓
Schema validation
↓
CartService
↓
Response

Route içerisinde business logic yazılmamalıdır.

Örneğin route içinde:

```
if existing_item:
    quantity += ...
```

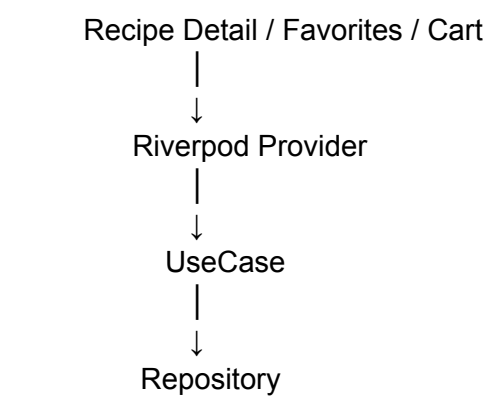
gibi business logic tutulmamalıdır.

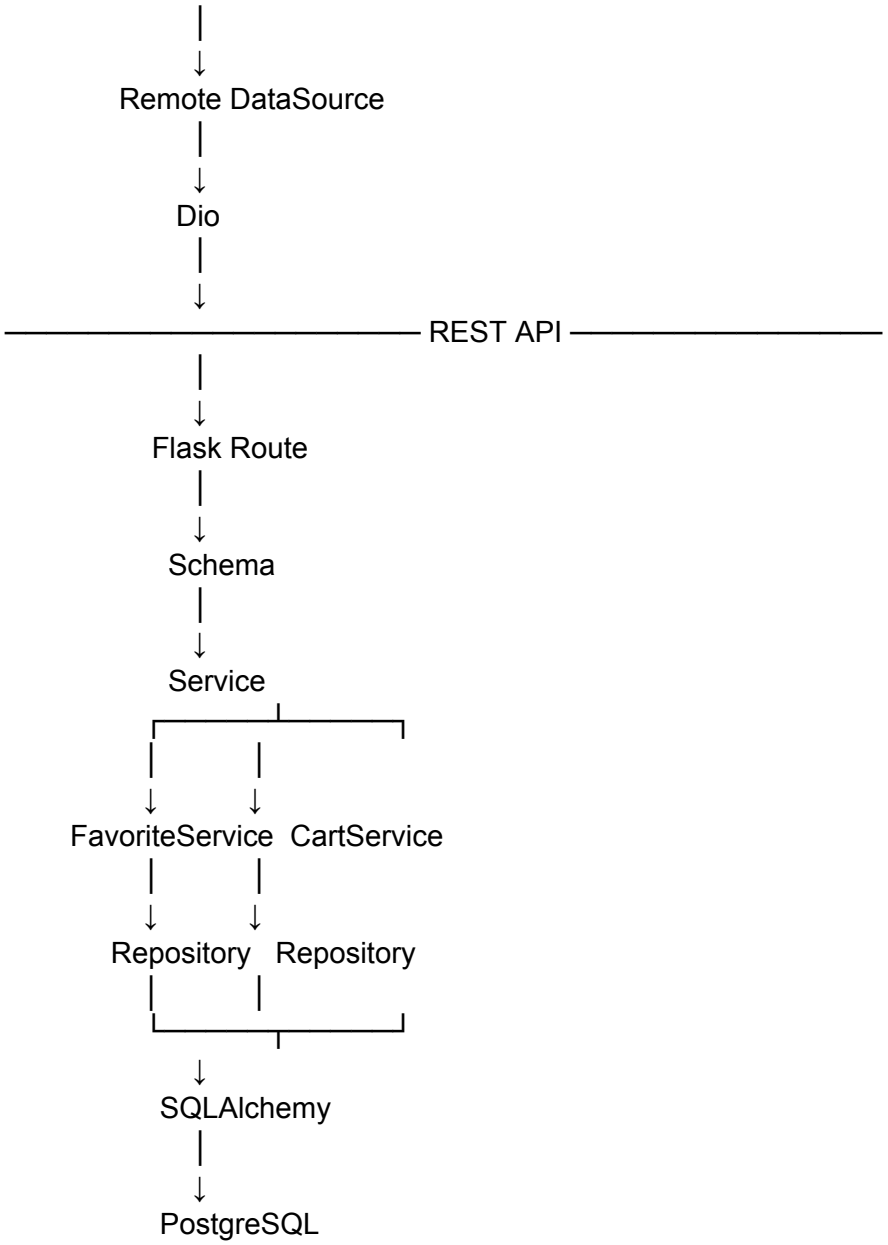
Bu mantık `CartService` içerisinde olmalıdır.

---

76. Favorite ve Cart Modülünün Genel Mimari Akışı

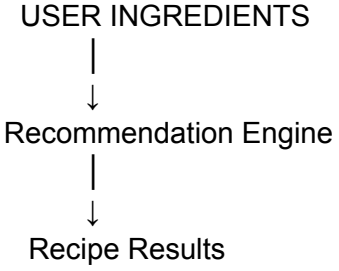
FLUTTER

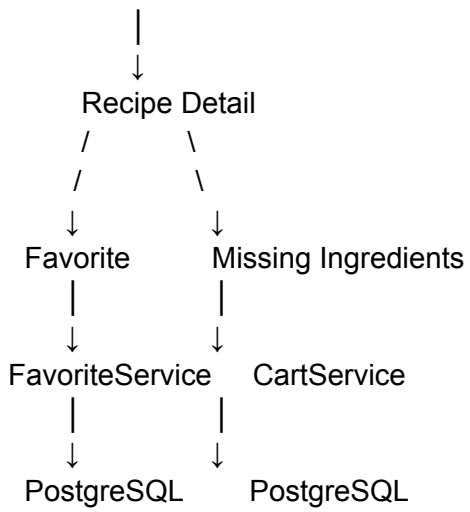




---

77. Recipe Matching + Favorites + Cart Genel Akışı





---

78. Modüller Arası Sorumluluk Sınırları

Recipe Matching

Sorumlu olduğu konular:

Match %
Matched ingredients
Missing ingredients
Recipe ranking

Favorites

Sorumlu olduğu konular:

User ↔ Recipe favorite relationship

Cart

Sorumlu olduğu konular:

User ↔ Ingredient shopping items
Quantity
Unit
Merge

Bir modül diğerinin business logic'ini üstlenmemelidir.

---

79. MVP Son Durum

MVP sonunda kullanıcı:

1. Malzemelerini seçer
   ↓
2. Tarif önerilerini görür
   ↓
3. Tarifi açar
   ↓
4. Match percentage görür
   ↓
5. Eksik malzemeleri görür
   ↓
6. Eksik malzemeleri sepete ekler
   ↓
7. Sepeti düzenler
   ↓
8. Tarifi favoriler
   ↓
9. Favorilerinden tekrar açar

akışını tamamlayabilmelidir.

---

80. Future Extensions

Favorites için gelecekte:

- Favorite collections
- Tarif kategorilerine göre favoriler
- Recently viewed recipes
- Favorite synchronization
- Offline favorite cache

eklenebilir.

Cart için:

- Birden fazla alışveriş listesi
- Liste paylaşımı
- Offline cart

- Cart synchronization
- Market entegrasyonu
- Market bazlı ürün eşleştirme
- Fiyat karşılaştırma
- Alışveriş hatırlatıcıları
- Tekrarlanan alışveriş listeleri
- E-commerce entegrasyonu

eklenebilir.

Ancak bu özellikler MVP'ye dahil edilmemelidir.

---

81. MVP Dışı Özellikle Korunması Gereken Sınırlar

Aşağıdaki işlemler Cart modülüne eklenmemelidir:

Payment
Order creation
Real-time market price
Payment provider
Delivery tracking

Cart şu aşamada yalnızca:

Shopping List / Ingredient Management

amacıyla kullanılmalıdır.

---

82. Final Teknik Kurallar

Bu modül için temel kurallar:

1. Favorites kullanıcıya özeldir.
2. Cart kullanıcıya özeldir.
3. Kullanıcı kimliği JWT'den alınır.
4. Client tarafından gönderilen `user_id` güvenilir kabul edilmez.
5. Aynı tarif aynı kullanıcı tarafından iki kez favorilenemez.
6. Aynı ingredient + unit kombinasyonu aynı kullanıcıda merge edilir.

7. Farklı unit'ler otomatik olarak birbirine dönüştürülmez.
8. Quantity sıfır veya negatif olamaz.
9. Silme işlemi DELETE endpointleriyle yapılır.
10. Backend gerçek veri kaynağıdır.
11. Optimistic UI kullanılabilir ancak rollback mekanizması bulunmalıdır.
12. Repository yalnızca veri erişiminden sorumludur.
13. Service business logic'ten sorumludur.
14. Route business logic içermez.
15. Schema validation route ile service arasındaki sınırda uygulanır.
16. Ownership backend tarafından kontrol edilir.
17. Database unique constraint duplicate kayıtları engeller.
18. Cart merge transaction-safe olmalıdır.
19. Recipe Matching eksik malzemeyi belirler; Cart bu malzemeyi yönetir.
20. MVP'de gerçek market siparişi ve ödeme bulunmaz.

---

83. Document 12 Sonuç

Smart Chef & Pantry AI uygulamasında Favorites ve Cart modülleri, kullanıcı deneyiminin kişiselleştirilmesini sağlayan ancak mimari açıdan birbirinden ayrılmış iki temel kullanıcı modülüdür.

Favorites:

User → Recipe

ilişkisini yönetirken,

Cart:

User → Ingredient → Quantity → Unit

ilişkisini yönetir.

Her iki modül de JWT authentication ve ownership kontrolü ile korunur.

Cart tarafında özellikle:

same user
+
same ingredient
+
same unit
=
same cart item

kuralı uygulanır ve yeni miktar mevcut miktara eklenir.

Recipe Matching Engine ise Cart'ın içinde yer almaz. Matching Engine eksik malzemeleri belirler, CartService ise bu malzemeleri kullanıcı sepetine dönüştürür.

Sonuç olarak modül mimarisi:

Flutter
↓
Riverpod
↓
UseCase
↓
Repository
↓
RemoteDataSource
↓
Dio
↓
Flask REST API
↓
Schema
↓
Service
↓
Repository
↓
SQLAlchemy
↓
PostgreSQL

şeklinde korunmalıdır.

Bu yapı, MVP'nin basit tutulmasını sağlarken ileride offline kullanım, market entegrasyonu, fiyat karşılaştırması, alışveriş listeleri ve kişiselleştirilmiş öneriler gibi özelliklerin mevcut mimariyi bozmadan eklenebilmesine olanak sağlar.